Database Bottlenecks

diagnose what's saturated, then climb the scaling ladder

Phase 3 videos on YouTube

In one line: when the database is the bottleneck, first find what is saturated (connections, CPU, disk IOPS or locks), watch for hot rows, then climb the ladder in order: optimise queries, add indexes, pool connections, cache hot reads, add read replicas, partition tables, and only then shard.

1 Adding servers stops helping

Your app servers scale out easily: add more, and throughput goes up. Then one day adding servers makes nothing faster. Latency climbs, and every graph points to the same place: the database. It's the most common bottleneck in system design.

2 What runs out



Figure 1. The four things a database runs out of.

- **Connections**: each Postgres connection is a process using several MB; thousands of them slow everything (Phase 2, lesson 82 covers pooling).
- **CPU**: expensive queries, sorts, hash joins, sequential scans.
- **Disk IOPS**: random reads that miss the buffer cache, and writes to the WAL.
- **Lock contention**: transactions waiting on the same rows.

Look at `pg_stat_activity` (what's running and waiting), `pg_stat_statements` (which queries cost the most total time), CPU and IOPS graphs, and lock waits.

3 The sneaky one: the hot row

Every request updates the same row, like a global counter. Each update must lock the row, so updates run **one after another**, no matter how big the database is. Throughput is capped by how fast one row can be locked, changed and committed: maybe a few hundred to a few thousand per second.

WORKED EXAMPLE: THE PAGE-VIEWS COUNTER

A `page_views` row is updated on every request. At 5,000 requests per second, transactions queue behind its lock and latency explodes. Fixes:

- **Sharded counters**: 64 rows (`shard = random 0..63`), increment one at random, `SUM` on read. Contention drops 64 times.
- **Buffer in Redis**: `INCR`, and flush to the database every few seconds (write-behind, lesson 18).
- **Events**: send view events to a queue and aggregate them in batches.

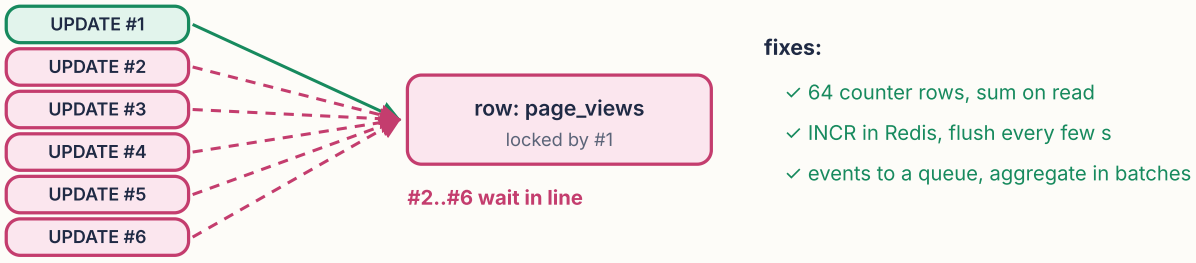


Figure 2. A hot row serialises every writer; spreading or batching the writes removes the queue.

4 The scaling ladder

Start with the cheapest steps:

1. **Optimise queries**: `EXPLAIN ANALYZE`, fix N+1 queries, select only needed columns, paginate with cursors.
2. **Add the right indexes** (composite, covering, partial: Phase 2, database lessons).
3. **Connection pooler** such as PgBouncer, so thousands of app connections share a few dozen database connections.
4. **Cache hot reads** in Redis to cut read load (lessons 15 to 33).
5. **Read replicas**: route read-only queries to them, accepting a little replication lag.
6. **Partition big tables** by time or key, so queries and maintenance touch less data.
7. **Shard** across several databases: scales writes, but costs cross-shard queries, transactions and operational complexity.

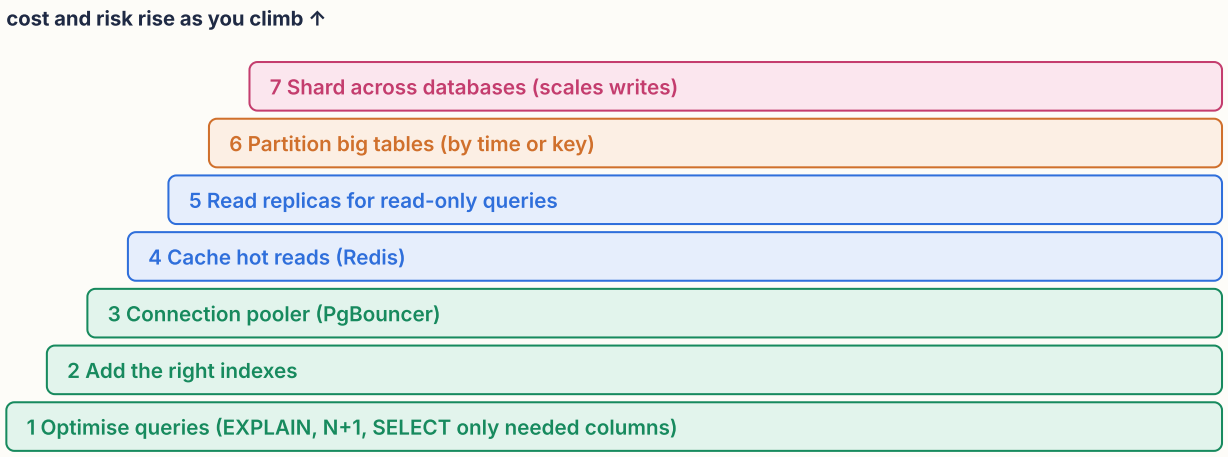


Figure 3. The database scaling ladder: climb from the bottom, measuring after each step.

Climb in order. Each step is cheaper and less risky than the next. Many teams jump to sharding when a missing index or a connection pooler would have solved the problem. And measure after each step: the

bottleneck may move somewhere else.

COMMON MISTAKES

- Scaling the database up before looking at the top queries in pg_stat_statements.
- Adding read replicas when the problem is writes or locks.
- Autoscaling app servers that each open 20 connections, until the database hits max_connections.
- Sharding first, and spending a year on cross-shard queries.

5 Key terms

Saturation: a resource at its limit, so work queues

Sharded counter: a counter split over many rows

Connection pooler: many app connections → few DB connections

Hot row: one row updated by most requests

pg_stat_statements: Postgres query cost statistics

Read replica: a copy that serves read-only queries

6 Interview questions

Q1. What is your step-by-step approach when the database is the bottleneck? BASIC

First find what's saturated: connections, CPU, IOPS or locks. Then climb the ladder: optimise queries, add indexes, pool connections, cache hot reads, add read replicas, partition tables, and only then shard.

Q2. How do you fix a hot row? BASIC

Split it into sharded counters, buffer increments in Redis, or aggregate events in batches.

Q3. CPU is low but queries are slow and p99 is terrible. What do you suspect? SCENARIO

Lock contention (hot rows, long transactions) or I/O waits; check pg_stat_activity wait events and lock waits, and look for long-running transactions holding locks.

Q4. Reads are 95% of traffic and the primary is at 90% CPU. Next steps? SCENARIO

Find and optimise the top read queries and indexes, then cache hot reads in Redis, then route remaining read-only queries to replicas, keeping read-your-writes paths on the primary.

Q5. Why can adding app servers make the database slower? TRICKY

More servers mean more connections and more concurrent queries; past a point, context switching, memory per connection and lock contention reduce database throughput. A pooler caps concurrency.

Q6. When is sharding actually the right step? TRICKY

When writes or data size exceed what one well-tuned primary (with partitioning) can handle, and you have a shard key that keeps most queries and transactions on one shard.

Next: SD-J-004 · Asynchronous Processing backendroadmap.com